Tests y chequeos de seguridad integrados al repositorio

TC5061 — Operaciones de aprendizaje automático (Gpo 10) · Actividad 7

Rolando Calderón Carballo · A01840537 · Trabajo individual

Repositorio: github.com/a01840537/TC5061-Proyecto-Equipo32-rolando · **Fecha:** 31 de octubre de 2026

1. Alcance

Esta actividad integra al repositorio del proyecto MiBici cuatro mecanismos de calidad y seguridad, todos ejecutables con un comando y documentados en el README:

- **Pruebas automatizadas** con pytest: 26 pruebas en 4 suites (unitarias, casos límite y reproducibilidad).
- **Validación de datos** con DeepChecks: integridad y deriva (drift), integrada como entry point `validate` del MLflow Project.
- **Auditoría de dependencias** con pip-audit: CVEs conocidos, remediación y riesgo aceptado.
- **Análisis estático de seguridad** del código con bandit.

Todo el trabajo se integró vía rama `feat/tests-y-seguridad` y pull request a `main` (PR #2), con las versiones de las herramientas pineadas en `requirements.txt`.

2. Pruebas automatizadas (pytest)

Comando: `python -m pytest tests -v` · Resultado: **26 pruebas aprobadas**.

Suite	Pruebas	Qué cubre
`test_features_p1.py`	3	No-fuga temporal de los rezagos de P1: panel contiguo desde la primera actividad, rezagos solo con días anteriores, rezago menor al horizonte lanza error (entrega 1).
`test_dataset.py`	10	Ingesta: normalización de columnas con acentos y sus variantes históricas («Año_de_nacimiento» → `anio_nacimiento`), fechas inválidas → NaT sin romper, minimización de datos personales, lectura con fallback utf-8 → latin-1, y hash SHA-256 del versionado de datos determinista y sensible a un solo byte.
`test_features_p2.py`	6	Etiqueta de riesgo de P2 con viajes sintéticos: una ráfaga de 8 viajes marca `sin_bicis` en la estación origen y `sin_anclajes` en la destino. Casos límite: estación con capacidad 0 descartada, filas sin ventana futura completa descartadas, horizonte fuera de [1, 24) lanza `ValueError`.

`test_reproducibilidad.py`	5	Misma semilla ⇒ predicciones *idénticas* (no solo parecidas) con las fábricas reales de modelos: P1 (HGB y Poisson), P2 (probabilidades) y P3 (KMeans), más un control negativo que verifica que una semilla distinta sí cambia el resultado.

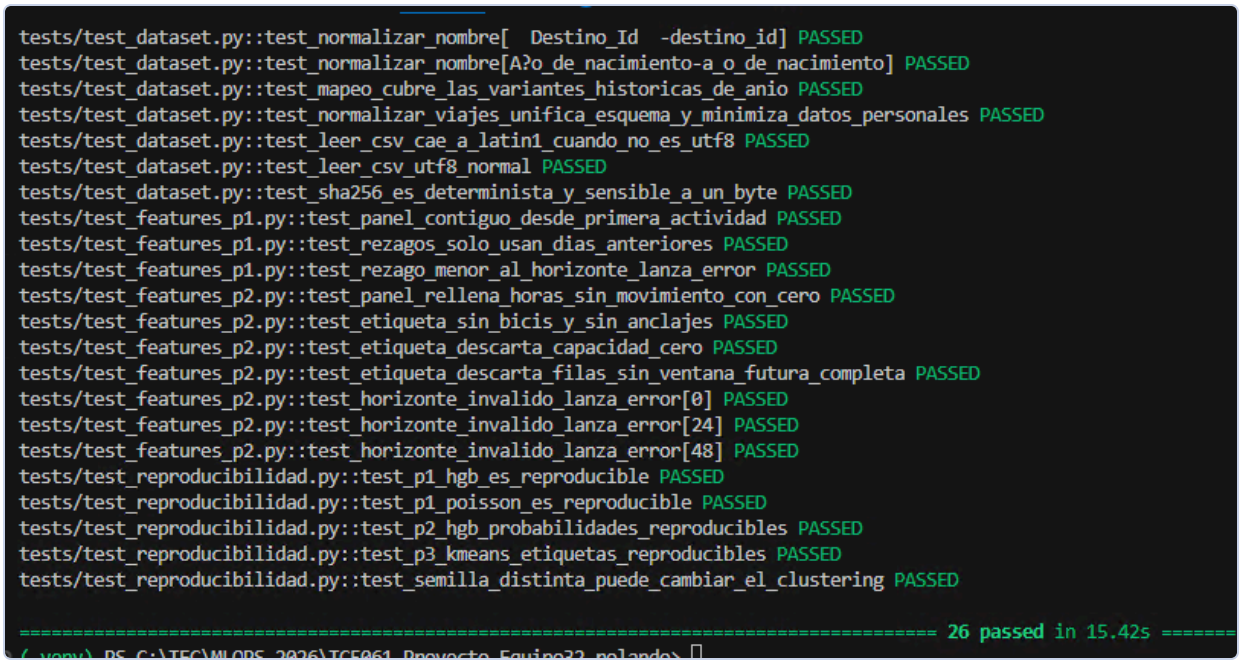


Figura 1. Corrida completa de la suite: 26 pruebas aprobadas.

3. Validación de datos (DeepChecks)

El módulo `mibici/data_validation.py` corre dos suites de DeepChecks sobre la tabla de modelado de P1 y quedó integrado como entry point del MLflow Project:

```
mlflow run . -e validate --env-manager local
# equivalente: python -m mibici.data_validation --sample 50000
```

- `data_integrity` **sobre TRAIN:** 12/12 chequeos aprobados — sin duplicados, nulos mezclados, columnas constantes ni conflictos de tipo.
- `train_test_validation` **TRAIN vs TEST:** 11/12 — Feature Drift en `dias_activa` (KS ≈ 0.99).

> **Interpretación del drift detectado:** `dias_activa` cuenta los días transcurridos desde la primera actividad de cada estación. En TEST (meses posteriores del año) toda estación lleva más tiempo activa que en TRAIN, así que el desplazamiento de la distribución ocurre *por construcción*: es deriva determinística de una feature temporal, no un problema de calidad de datos. El chequeo funcionó exactamente como debe — detectó el cambio y el equipo lo explica en lugar de ignorarlo.

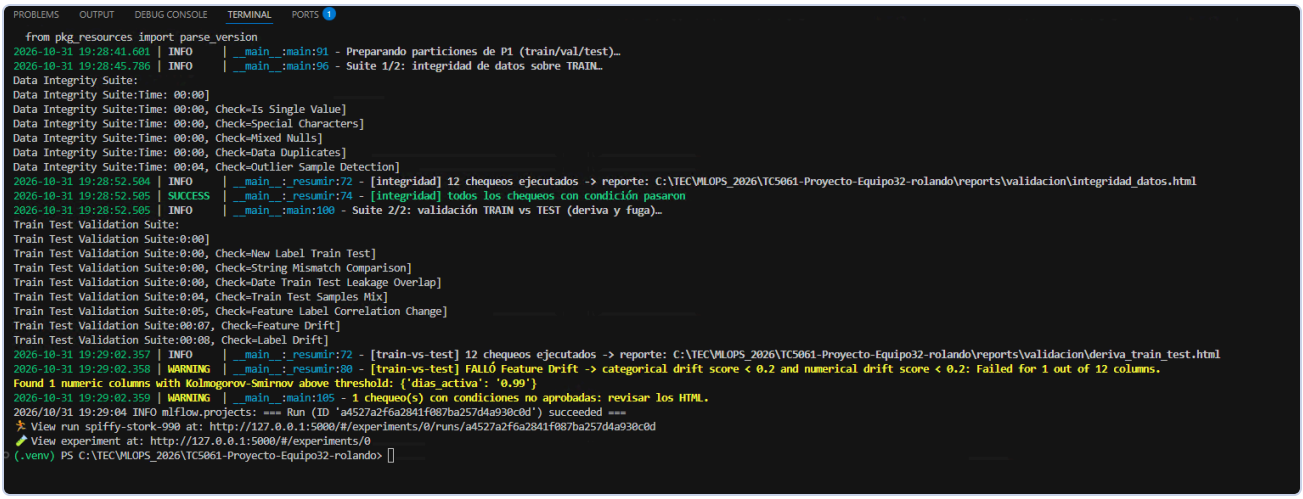


Figura 2. Resumen en consola: integridad 12/12 aprobados y un único fallo de Feature Drift en TRAIN vs TEST; reportes HTML generados en `reports/validacion/`.

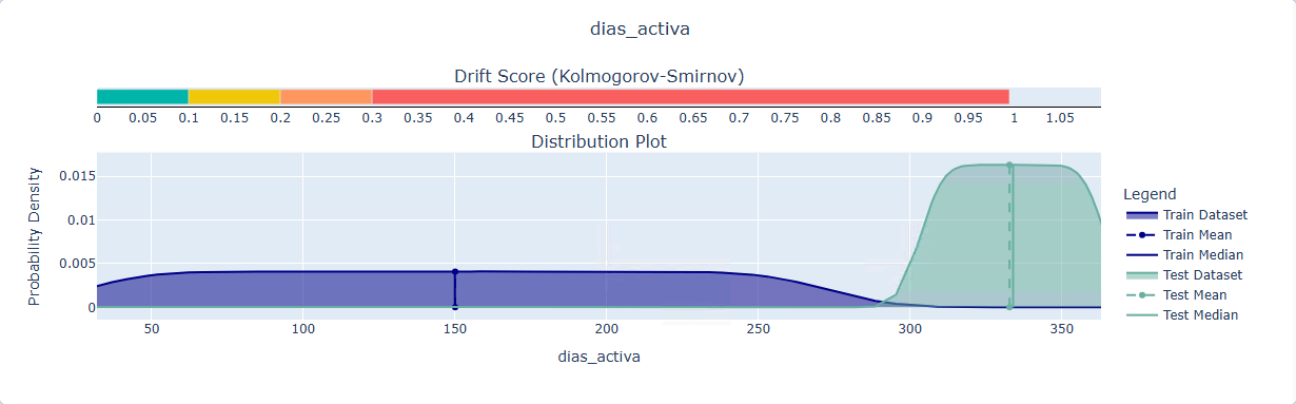


Figura 3. Reporte HTML de DeepChecks: distribución de `dias_activa` en TRAIN (0–300 días) vs TEST (concentrada en 300+), con score Kolmogorov-Smirnov de 0.99.

Compatibilidad documentada: DeepChecks 0.19.1 requirió tres ajustes para convivir con el entorno del proyecto (scikit-learn 1.9, NumPy 2, Python 3.14), documentados en el propio módulo: reinscribir el scorer `max_error`, restaurar el alias `np.Inf` y pinear `setuptools<81` (provee `pkg_resources`).

4. Auditoría de dependencias (pip-audit)

Comando: `pip-audit` sobre el entorno virtual del proyecto.

	Hallazgos	Detalle
Antes	86 vulnerabilidades en 12 paquetes	Casi todas en dependencias transitivas (aiohttp, certifi, filelock, gitpython, pillow, tornado, werkzeug…); una en dependencia directa: mlflow 3.17.1 (CVE-2026-53995).
Remediación	—	Actualización de mlflow a 3.17.2 y de las transitivas afectadas a sus versiones con fix; re-ejecución de la suite completa de pruebas (26/26 en verde) para confirmar que nada se rompió.
Después	3 hallazgos en 1 paquete	Todos en setuptools 80.10.2 — **riesgo aceptado** (ver abajo).

> **Riesgo aceptado — setuptools:** el fix exige setuptools ≥ 83, pero DeepChecks 0.19.1 depende de `pkg_resources`, retirado de setuptools ≥ 81 (con setuptools 84 instalado, `import deepchecks` falla con `ModuleNotFoundError`). Al tratarse de una herramienta de desarrollo en entorno local, sin servicios expuestos ni instalación de paquetes no confiables, se acepta el riesgo, se pinea `setuptools<81` en `requirements.txt` y se re-evaluará cuando DeepChecks publique soporte para setuptools modernos.

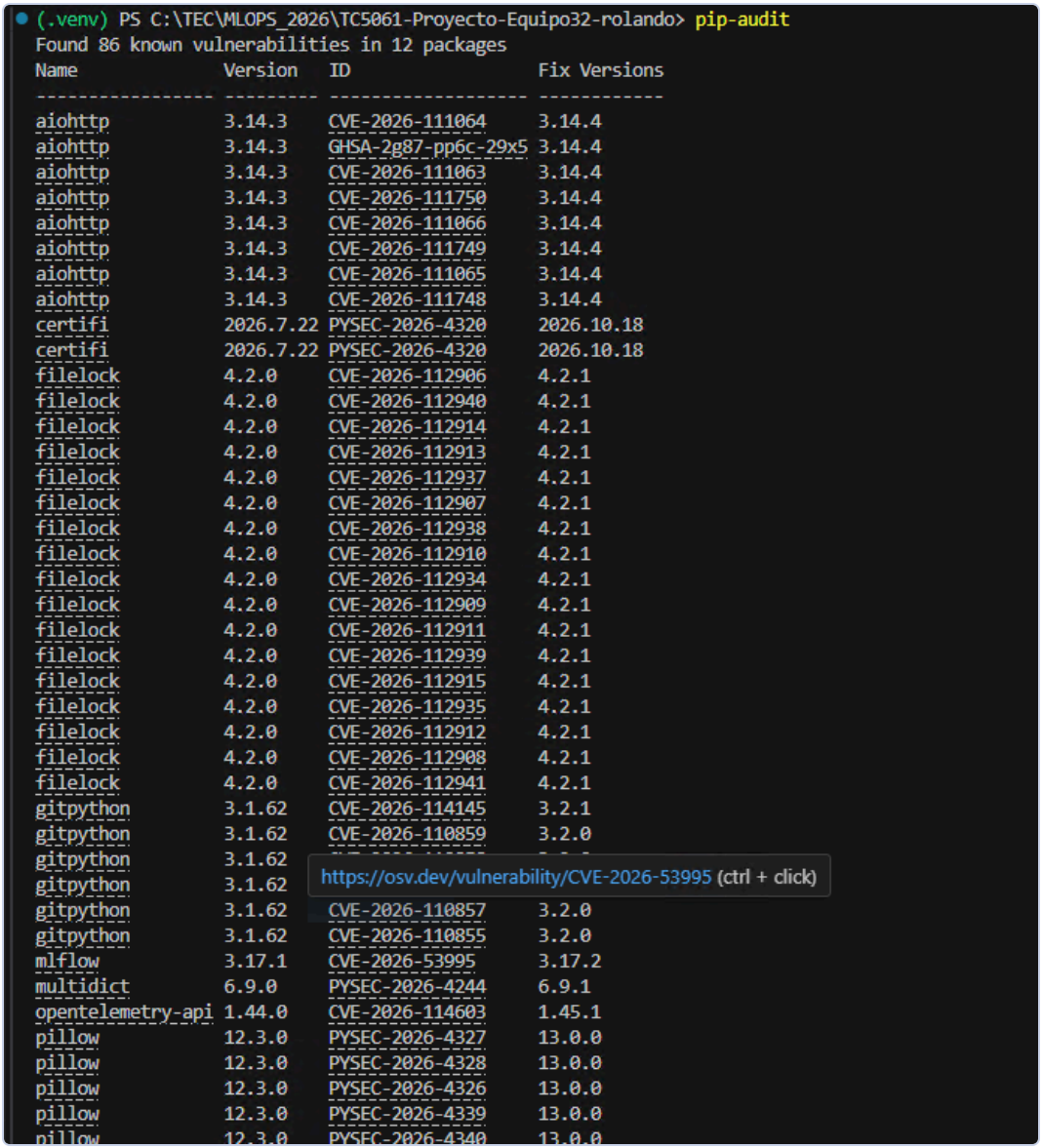


Figura 4. pip-audit antes de remediar: 86 vulnerabilidades conocidas en 12 paquetes (fragmento).

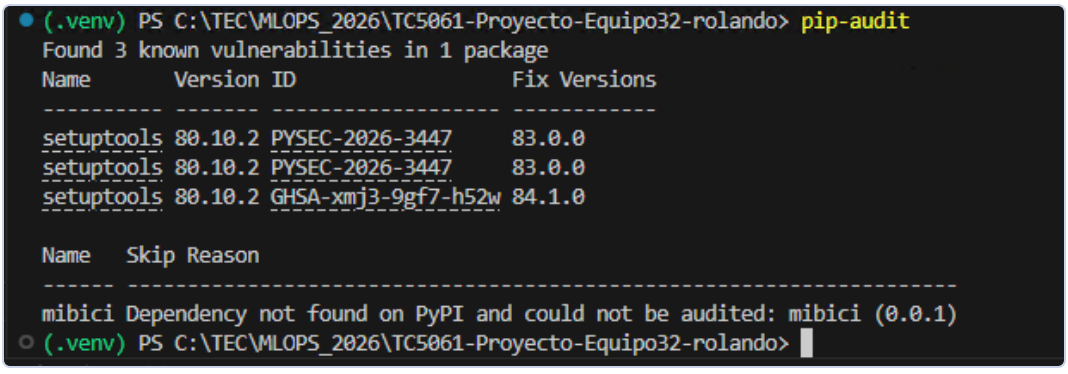


Figura 5. pip-audit después de remediar: solo los 3 hallazgos de setuptools, documentados como riesgo aceptado.

5. Análisis estático de seguridad (bandit)

Comando: `bandit -r mibici` · Resultado: **0 hallazgos Medium/High** y 13 Low en 2,926 líneas de código.

Los 13 hallazgos Low son un único patrón repetido en `p1_regresion.py`, `p2_clasificacion.py` y `p3_clustering.py`: las llamadas a `git rev-parse HEAD` / `git status` con las que el proyecto captura el commit para el linaje de los runs de MLflow (reglas B404, B603 y B607, todas CWE-78).

> **Disposición — aceptados sin cambios:** el comando se invoca como lista fija de argumentos (`["git", "rev-parse", "HEAD"]`), sin entrada del usuario y sin `shell=True`, que es precisamente la forma segura que la documentación de bandit recomienda para subprocess. No hay superficie de inyección.

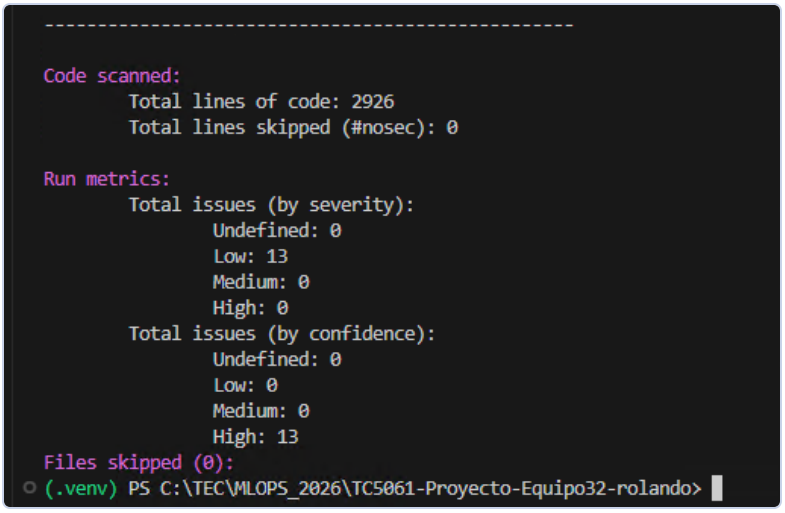


Figura 6. Resumen de bandit: 2,926 líneas analizadas, 0 Medium/High, 13 Low (patrón subprocess/git aceptado).

6. Higiene del repositorio y cierre

- `.gitignore` excluye credenciales y datos: `.env`, `data/`, `mlflow.db`, `mlruns/`, `mlartifacts/`, modelos serializados y los reportes pesados de validación (`reports/validacion/`, ~40 MB por HTML).
- Versiones pineadas en `requirements.txt`: `deepchecks==0.19.1`, `bandit==1.9.4`, `pip-audit==2.10.1`, `setuptools<81`, `mlflow==3.17.2`.
- Integración por rama y pull request (`feat/tests-y-seguridad` → PR #2 → `main`), con la evidencia y las instrucciones de ejecución en la sección «Actividad 7» del README.

Con esto el repositorio queda con una red de seguridad ejecutable en tres comandos — `pytest`, `mlflow run . -e validate` y `pip-audit` / `bandit` — que cubre corrección del código, calidad de los datos y seguridad de las dependencias.